rapport_projet_final_python

STEPKOWSKI Maxime
CHAMPION Enrike
MASSION Côme

Lien du répertoire git : https://github.com/MaximeStepkowski/log-analysis

Ingestion en flux

Pour traiter de gros fichiers sans saturer la RAM, l'ingestion (`log_engine/ingestion.py`) s'appuie sur des générateurs Python. `read_logs` ouvre un fichier et renvoie ses lignes au fur et à mesure avec `yield`, sans jamais le charger en entier. Côté performances, on limite les entrées-sorties avec un tampon de lecture de 1 Mo et une lecture par lots d'environ 4 Mo, tout en gardant un flux ligne par ligne. Le retrait du saut de ligne passe par `map` et une méthode native, ce qui évite une boucle Python explicite.

Nous avons ensuite ajouté `read_logs_from_directory`, qui parcourt les fichiers correspondant à un motif, les trie et les enchaîne avec `yield from`. Elle permet d'analyser plusieurs fichiers comme un seul flux, sans construire une liste contenant toutes les lignes. Elle vérifie aussi que le chemin est un dossier et qu'au moins un fichier correspond au motif. Dans le cas contraire, elle lève une erreur explicite plutôt que de lancer une analyse vide ou ambiguë.

Cette conception permet de traiter des fichiers très volumineux avec une empreinte mémoire bornée, de l'ordre de quelques Mo, indépendante du nombre total de lignes.

Génération des logs

Pour tester le moteur, il nous fallait beaucoup de logs réalistes, donc on a écrit notre propre générateur (`log_generator/generate_logs.py`). Il est configurable : nombre de lignes, numéro du fichier de sortie, cadence maximale et taille des lots d'écriture. Chaque ligne suit l'un de trois formats (combined enrichi du port client, JSON ou clé-valeur), avec des distributions pondérées pour les méthodes et les codes de statut. Pour éprouver la robustesse du moteur, il injecte aussi des lignes vides ou corrompues, des IP en blacklist et une simulation d'attaque par bruteforce sur `/admin/login`. Les timestamps partent d'une date aléatoire et avancent selon un débit en logs par seconde, ce qui rend les données plus crédibles.

Dans `log_generator/generate_multiple_logs.py`, nous avons ajouté la génération de plusieurs fichiers dans un dossier. Le débit temporel est partagé et chaque fichier reprend

là où le précédent s'est terminé : l'ensemble forme une chronologie continue, dont la durée est cohérente avec le nombre total de lignes.

Ces générateurs permettent de tester aussi bien un petit jeu de données pour les tests unitaires que plusieurs millions de lignes pour les essais de performance.

Parsing et construction des enregistrements

Dans `log_engine/parsing.py`, nous avons pris en charge cinq formats de logs : les objets JSON, les logs système au format clé-valeur ou brut, et les logs HTTP au format Combined ou clé-valeur. Chaque ligne est testée dans cet ordre : JSON si elle commence par `{`, puis les motifs HTTP, puis les motifs système. Chaque format est analysé par une expression régulière ou par le parseur JSON, puis converti en dictionnaire. Tous partagent un noyau commun de champs (`timestamp`, `level`, `user`, `payload`, `connection`), enrichi pour HTTP (méthode, chemin, statut, taille, durée, etc.). Les enregistrements passent ensuite par `normalize_record`.

La construction des données est centralisée dans `build_record` et `build_http_record`. Elles convertissent les dates en objets `datetime` avec fuseau horaire (UTC par défaut quand il est absent), valident les adresses IP avec le module `ipaddress`, convertissent ports, statuts, tailles et durées en entiers, et regroupent la connexion dans un tuple immuable `(IP, port, protocole)`. `build_record` contrôle en plus le niveau de sévérité (`INFO`, `WARNING`, `ERROR`, `CRITICAL`), tandis que `build_http_record` le déduit du statut de la réponse (≥ 500 : `CRITICAL`, ≥ 400 : `ERROR`, ≥ 300 : `WARNING`, sinon `INFO`).

Nous avons ajouté un cache `lru_cache` sur la conversion des dates et la validation des adresses IP : quand une même valeur revient, le résultat déjà calculé est renvoyé sans réexécuter la fonction.

Le parallélisme est venu d'un problème très concret. Au départ, tout tournait de façon séquentielle, mais avec des logs générés à un débit de 40 000 lignes par seconde, l'exécution de nos tests est devenue beaucoup trop longue. Comme dans nos précédents projets (SAE) du BUT, où l'on avait déjà cherché à optimiser la lecture de gros volumes, on a voulu faire pareil ici et répartir le travail sur des workers. C'est le rôle de `parse_logs_parallel` : elle répartit le parsing sur plusieurs processus par lots de lignes. L'ordre des lignes est conservé, donc la sortie est identique à celle du parsing séquentiel. Avec un worker, la fonction revient automatiquement au séquentiel. Point à garder en tête : chaque lot doit être envoyé aux processus puis récupéré, donc le gain dépend du coût du parsing par rapport à ce transfert.

Normalisation et transformations

Le module `log_transformation/normalization.py` uniformise les enregistrements une fois parsés. `normalize_record` est une fonction qui travaille sur une copie et ne modifie jamais l'enregistrement d'origine. Elle réduit les espaces superflus dans les valeurs textuelles, met en majuscules le niveau, le protocole et la méthode HTTP, convertit le statut, la taille et la durée en entiers, normalise le tuple de connexion (IP, port, protocole) et nettoie le payload. Pour les logs système, le nom d'utilisateur est en plus mis en minuscules. Les enregistrements invalides traversent cette étape sans erreur. Cette étape nettoie les valeurs textuelles, harmonise les champs attendus et préserve les enregistrements invalides sous une forme exploitable. Nous avons aussi conservé les modules de transformation dédiés au filtrage et au suivi, afin de séparer la lecture, la conversion et l'analyse.

`parse_logs` applique cette normalisation avec `map`, dans un flux de générateurs : chaque ligne est lue, parsée puis normalisée à la demande, sans liste intermédiaire. Le même enchaînement est utilisé dans les processus workers du parsing parallèle. Les modules `parsing.py` et `normalization.py` ont besoin l'un de l'autre : le premier utilise la fonction `normalize_record`, et le second utilise le type `Record` défini dans le premier. Si les deux s'importaient normalement, Python se retrouverait dans une boucle et lèverait une erreur au démarrage (import circulaire). Pour l'éviter, l'import de `Record` est placé sous `TYPE_CHECKING` : il n'est lu que par les outils de vérification de types (comme mypy ou l'éditeur), jamais lors de l'exécution du programme. Cette séparation entre lecture, conversion et analyse permet de faire évoluer chaque étape indépendamment.

Filtrage des enregistrements

Dans `log_transformation/filter.py`, la fonction `valid_log` est le prédicat qui décide si un enregistrement est exploitable. Elle vérifie que l'élément est un dictionnaire, que l'horodatage est présent et non vide, que le niveau fait partie de `INFO`, `WARNING`, `ERROR` et `CRITICAL`, que l'utilisateur est une chaîne, que le payload est un dictionnaire ou une chaîne, et que la connexion est un tuple de trois éléments (IP non vide, port entier entre 1 et 65 535 hors booléens, protocole non vide).

Dans `log_transformation/tracker.py`, la classe `Tracker` est un objet appelable utilisé comme prédicat de `filter` : `main.py` l'emploie dans `filter(lambda record: tracker(record), parsed_stream)`. Pour chaque enregistrement, elle le route avec `route_log`, mémorise le premier exemple de chaque catégorie, compte les lignes lues, rejetées et inconnues, conserve quelques échantillons (trois par défaut) puis renvoie le résultat de `valid_log`. Le filtrage reste donc paresseux : les lignes invalides sont comptées et échantillonnées au passage, sans liste intermédiaire.

Routage et gestion des cas invalides

Dans `log_engine/routing.py`, nous avons utilisé le pattern matching `match-case` pour orienter chaque élément selon sa structure (dictionnaire, chaîne ou tuple) et son contenu. `route_log` renvoie un couple (catégorie, description) parmi huit catégories : `invalid`, `http_error`, `http`, `dict` (log système avec niveau), `ignored`, `raw`, `connection` et `unknown`. L'ordre des `case` est déterminant, car le premier motif qui correspond l'emporte : les cas spécifiques (ligne rejetée, erreur HTTP) précèdent les cas généraux (simple niveau de sévérité). Les motifs combinent dictionnaires, alternatives, gardes et séquences.

Les tuples de connexion sont validés (adresse IP, port entre 0 et 65 535, protocole non vide, avec exclusion des booléens). Un tuple invalide est routé vers `invalid` et toute structure non reconnue vers `unknown`. Le routage ne s'interrompt jamais : chaque élément reçoit une catégorie et un message explicite, que le programme principal peut compter et afficher. Les autres cas limites sont traités en amont : dossier introuvable ou sans fichier correspondant dans `ingestion.py`, et ligne vide, date ou IP invalide ou champ manquant dans `parsing.py`, qui produit alors un enregistrement invalide plutôt que d'arrêter l'analyse.

Ensembles d'IP et liste noire

Dans `log_processing/ip_sets.py`, `load_blacklist` lit `data/blacklist.txt` (une IP par ligne, `#` pour les commentaires), valide chaque adresse avec `ipaddress` et renvoie un `frozenset` : la liste noire ne peut plus être modifiée une fois chargée. Une adresse invalide lève une erreur qui indique le numéro de la ligne fautive.

`IPCollector` accumule deux `set` au fil du flux : les IP actives et les IP ayant généré des erreurs (niveau `ERROR` ou `CRITICAL`). Elle lit la connexion de chaque enregistrement avec un `match-case` et ignore ceux qui n'en ont pas. Le générateur `track_ips` s'insère dans le pipeline sans le modifier : il alimente le collecteur puis renvoie chaque enregistrement. La mémoire dépend ainsi du nombre d'IP distinctes, pas du nombre de lignes.

`compare_with_blacklist` applique les quatre opérations d'ensembles demandées entre la liste noire et les IP actives : intersection `&` (IP blacklistées qui ont contacté le serveur), différence `-` dans les deux sens (blacklistées silencieuses, actives non blacklistées), union `|` (toutes les IP connues) et différence symétrique `^` (IP présentes dans un seul des deux ensembles). Chaque résultat est converti en `frozenset` car le type d'une opération dépend de l'opérande de gauche (`set - frozenset` renvoie un `set`), et le résultat est regroupé dans `BlacklistReport`, une dataclass immuable. `format_ip_report` en produit le rapport texte.

Agrégation avec defaultdict

Dans `log_engine/aggregation.py`, la classe `CountReport` compte les enregistrements par catégorie de routage, par niveau de sévérité, par utilisateur et par couple

utilisateur/niveau. Les compteurs sont des `defaultdict(int)` (et un `defaultdict` de `defaultdict` pour le dernier), ce qui permet d'incrémenter une clé absente sans `KeyError` ni test préalable. La catégorie vient de `route_log`, ce qui relie l'agrégation au `match-case`. Un niveau ou un utilisateur vide est compté sous `<inconnu>`, et une ligne invalide n'alimente ni les niveaux ni les utilisateurs pour ne pas fausser les distributions. `most_common` trie les compteurs par fréquence décroissante pour l'affichage.

Mesure de la mémoire

Dans `log_engine/memory.py`, nous avons implémenté des fonctions pour mesurer la mémoire résidente courante et le pic de mémoire du processus. Sous Linux, la mémoire résidente est lue dans `/proc/self/statm` (nombre de pages multiplié par la taille de page) et le pic est obtenu avec `resource.getrusage`, dont la valeur est convertie en octets. Sous Windows, nous interrogeons `GetProcessMemoryInfo` avec `ctypes` pour obtenir le working set courant et maximal. L'import de `resource` est optionnel afin que le module reste importable sur les systèmes qui ne le fournissent pas.

Avec l'option `--memory`, `main.py` échantillonne la mémoire courante toutes les 10 000 lignes pour calculer une moyenne, puis relève le pic en fin de traitement. Les deux valeurs figurent dans le rapport d'exécution, ce qui permet de vérifier concrètement que les générateurs et les lots bornés maintiennent une consommation maîtrisée. Ces mesures portent sur le processus principal, les workers du parsing parallèle n'y sont pas comptabilisés.

Détection des attaques brute-force

Les séries d'échecs d'authentification HTTP sont détectées par `BruteForceDetector` (`log_process/brute_force.py`). Les statuts `401` et `403` comptent comme des échecs. Chaque adresse IP a son propre compteur : une réponse de statut inférieur à `400` le remet à zéro, tandis que les autres statuts (par exemple `404` ou `500`) sont ignorés. Le seuil par défaut est de trois échecs consécutifs et se règle à la création du détecteur.

Le détecteur s'utilise de deux façons. En flux, `process` analyse chaque enregistrement déjà parsé au fil de l'eau, avec une mémoire constante : c'est le mode employé par `main.py`. En lot, `detect_brute_force` accepte des lignes brutes ou des dictionnaires, les trie par horodatage pour respecter l'ordre réel des événements, puis les traite par cycles : une réussite sépare deux séries d'échecs d'une même IP. Dans les deux cas, un ensemble d'adresses déjà alertées évite de produire plusieurs alertes pour une même série. Chaque alerte contient l'adresse IP, le nombre d'échecs au moment où le seuil est atteint, le statut, la date et un message.

L'algorithme en lot combine les deux types de boucles demandés. Une boucle `while` parcourt les cycles de chaque IP et une boucle `for` parcourt les enregistrements d'un

cycle. `continue` ignore les statuts qui ne sont pas des échecs, `break` interrompt le cycle dès que le seuil est atteint et que l'alerte est émise, et la clause `else` de la boucle `for` ne s'exécute que si le cycle se termine sans atteindre le seuil : elle consigne alors la fin complète du cycle sans alerte.

Les lignes invalides ou sans champs HTTP sont ignorées (avec un avertissement en mode lot) et ne bloquent pas la détection. `write_report` produit le rapport texte et crée le dossier de sortie si nécessaire. Enfin, `brute_force_cli.py` permet de lancer la détection en ligne de commande en choisissant le dossier de logs, le motif des fichiers et le fichier de sortie.

Dans `log_process/brute_force_cli.py`, nous avons exposé la détection avec une interface en ligne de commande. Les options permettent de choisir :

- le dossier d'entrée avec `--directory`
- le motif des fichiers avec `--pattern`
- le fichier du rapport avec `--output`

La CLI lit les fichiers du dossier, affiche les alertes dans le terminal et les écrit dans le rapport. La détection est aussi intégrée au pipeline principal (`main.py`), où elle exploite les enregistrements déjà lus et parsés sans relire les fichiers, son rapport se règle avec `--brute-force-output`.

Moteur de règles et audit

Dans `security/engine.py`, `apply_security_rules(*args, **kwargs)` reçoit les règles de sécurité comme fonctions de première classe dans `*args`, et l'enregistrement ainsi que les paramètres de configuration dans `**kwargs`. Chaque règle renvoie un résultat structuré (dictionnaire) ou `None`. La fonction vérifie que chaque règle est appelable et lève un `TypeError` si l'enregistrement est absent. L'état partagé entre les règles est porté par `SecurityContext` (liste noire, compteur d'erreurs HTTP par IP en `defaultdict`, IP déjà alertées, détecteur de force brute). Dans le pipeline, un chemin court évite de reconstruire les arguments pour chaque ligne.

Dans `security/rules.py`, trois règles sont fournies : `rule_blacklisted_ip` (IP présente dans la liste noire), `rule_http_errors` (seuil d'erreurs HTTP par IP, réglable avec `http_error_threshold` et fixé à 3 par défaut) et `rule_brute_force` (qui s'appuie sur `BruteForceDetector`). Toutes acceptent `**_` pour ignorer les paramètres qui ne les concernent pas : ajouter une règle ne demande aucune modification du moteur.

Dans `security/audit.py`, `collect_execution_audit` inspecte l'état d'exécution avec `globals()` et `locals()`, auxquels s'ajoutent des variables du pipeline fournies par `main.py`. Elle liste le nom et le type de chaque variable (hors noms commençant par `_`)

sans exporter les valeurs, pour éviter de divulguer des données ou de produire un rapport trop volumineux. `security/report.py` écrit le rapport texte : règles déclenchées, puis audit des portées. Il est activé avec `--security-report`.

Pipeline principal et menu

`main.py` enchaîne les générateurs en une seule passe : `read_logs_from_directory`, `parse_logs_parallel`, `filter` avec le `Tracker`, puis `track_ips`. La boucle finale alimente le `CountReport`, la détection de force brute ou les règles de sécurité, l'affichage de progression et l'échantillonnage mémoire. Seuls des compteurs, des ensembles et quelques échantillons sont conservés. À la fin, les rapports (comptage, mémoire, IP, force brute et, en option, sécurité) sont écrits dans `reports/`. Les erreurs de lecture de la liste noire ou d'écriture des rapports sont affichées avec un code de sortie non nul. Les options principales sont `--memory`, `--workers`, `--batch-size` et `--quiet`.

`log_analysis.py` fournit un menu interactif : génération de logs, lancement de `main.py` dans un sous-processus (dont la sortie est copiée dans `reports/last_run.txt`) et lecture des rapports page par page.

Environnement et livrables

Le projet s'exécute dans un environnement virtuel `venv`. Le fichier `requirements.txt` fige les versions des dépendances (`pytest` et ses dépendances : `iniconfig`, `packaging`, `pluggy`, `Pygments`) : le moteur lui-même n'utilise que la bibliothèque standard. Le `README.md` détaille l'installation sous Linux, macOS et Windows, l'utilisation des commandes et l'exécution des tests. Le code est organisé en packages (`log_engine`, `log_generator`, `log_transformation`, `log_processing`, `log_process`, `security`) avec `main.py` comme point d'entrée, sans notebook. Python 3.10 ou plus est nécessaire pour `match-case`.

Tests et validation

Nous avons écrit des tests unitaires avec `pytest` pour l'ingestion, les générateurs de logs, le parsing, la normalisation, la mesure mémoire, le filtrage et le suivi des enregistrements, le routage et les interfaces en ligne de commande, ... Les tests isolent les composants à l'aide de fichiers temporaires (`tmp_path`) et de remplacements de fonctions (`monkeypatch`), et couvrent les cas limites : dossier absent ou vide, ligne vide ou inconnue, valeurs invalides, arguments incorrects.

Les tests de détection de bruteforce vérifient le seuil d'alerte, la remise à zéro après une réussite, l'indépendance des compteurs entre plusieurs IP, l'ignorance des statuts autres que 401 et 403, et la lecture de formats mélangés depuis un dossier. La production du rapport est vérifiée dans les tests de la CLI.

Les modules ajoutés ensuite sont testés de la même façon : les opérations d'ensembles et le chargement de la liste noire (`test_ip_sets.py`), les compteurs `defaultdict` (`test_aggregation.py`), le moteur de règles et l'audit (`test_security.py`) et le menu interactif (`test_log_analysis.py`). La suite compte 78 tests, tous réussis.

Résultats et conclusion

Le pipeline a été exécuté sur trois fichiers générés (300 000 lignes, environ 67 Mo au total) avec `python main.py --memory` . Il lit 300 000 lignes, dont 297 567 enregistrements valides et 2 433 lignes invalides (lignes vides ou corrompues injectées par le générateur), en 4,6 secondes. Le pic de mémoire est d'environ 32 Mo (moyenne de 29,5 Mo), soit moins que la taille des fichiers : ils ne sont donc jamais chargés en entier.

Le comptage répartit les enregistrements en 259 311 requêtes `http` et 38 256 erreurs `http_error` , avec 229 764 `INFO` , 32 208 `ERROR` , 29 547 `WARNING` et 6 048 `CRITICAL` . L'analyse des IP relève 179 adresses actives, dont 3 sur les 5 de la liste noire, les 2 autres restant silencieuses. La détection de force brute produit 24 alertes. Avec les règles de sécurité activées, 8 440 résultats sont produits : 8 237 pour IP blacklistée, 179 pour seuil d'erreurs HTTP et 24 pour force brute.

Le projet couvre les quatre modules de l'énoncé : générateurs `yield` et `yield from` , parsing avec tuples immuables, `filter` , `map` et `match-case` , ensembles et `defaultdict` , détection de force brute avec `break` , `continue` et `else` , et moteur de règles avec `*args` , `**kwargs` , `globals()` et `locals()` . Chaque étape reste indépendante des autres, ce qui permet de faire évoluer le pipeline sans modifier l'ensemble.